

A
MAJOR PROJECT REPORT
ON
PINUX (Smart Home System)
Submitted in partial fulfillment for the award of the
DIPLOMA
IN
ELECTRICAL ENGINEERING

Submitted By

Priyanshu Raj (236991007)

Under the Guidance of

Mr. Gajendra Singh Rawat



**Department of Electrical Engineering
Faculty of Engineering and Technology
Gurukula Kangri (Deemed to be University), Haridwar-249404**

MAY, 2026

STUDENT DECLARATION

I hereby certify that the work is being presented in the major project entitled “PINUX (Smart Home System)” submitted in the Department of Electrical Engineering, Faculty of Engineering and Technology, Gurukula Kangri (Deemed to be University), Haridwar, is my own work carried out under the guidance of Mr. Gajendra Singh Rawat, Department of Electrical Engineering in partial fulfillment of the award of the Diploma in Electrical Engineering, Faculty of Engineering and Technology, Gurukula Kangri (Deemed to be University), Haridwar.

The matter presented in this report has not been submitted by me in anywhere for the award of any diploma or degree or to any other institute.

Name: Priyanshu Raj (236991007)

Signature: _____

CERTIFICATE

This is to certify that the Major project report entitled “PINUX (Smart Home System)” submitted by Priyanshu Raj (Reg No: 236991007), Diploma in Electrical Engineering, Faculty of Engineering and Technology, Gurukula Kangri (Deemed to be University), Haridwar, is a work carried out under the guidance of Mr. Gajendra Singh Rawat.

(Mr. Gajendra Singh Rawat)

(HOD Signature)

ACKNOWLEDGEMENT

I express my sincere gratitude to Mr. Gajendra Singh Rawat for his guidance, technical review, and constant support throughout the development of PINUX. I also thank the Department of Electrical Engineering for providing the academic environment needed to complete this project. The project improved my practical understanding of microcontrollers, cloud-connected systems, embedded control, and report preparation.

ABSTRACT

PINUX is a smart home control system designed around an ESP32 controller, Firebase Realtime Database, a web dashboard, and a Python-based automation layer. The system supports low-latency relay control, timer-based automation, and synchronized device state across the web interface and hardware layer. This corrected report documents the project architecture, hardware wiring, software design, testing procedure, limitations, and future scope. It also replaces incomplete discussion, missing figures, and placeholder text that were present in the earlier submission draft.

TABLE OF CONTENTS

Chapter 1: Project Introduction	1
Chapter 2: Hardware Design and Components	4
Chapter 3: System Architecture and Data Flow	8
Chapter 4: Backend Logic and Database Design	12
Chapter 5: Frontend Dashboard and User Experience	16
Chapter 6: Testing, Results, and Limitations	20
Chapter 7: Corrected Source Code Summary	23
Chapter 8: Conclusion and Future Scope	28
References	30

LIST OF FIGURES

Figure 1. PINUX 4-Layer System Architecture	9
Figure 2. PINUX Command and Feedback Flow	10
Figure 3. Hardware Wiring Overview	11
Figure 4. Dashboard Interface Screenshot	17
Figure 5. Firebase and Cloud Layout Reference	13
Figure 6. ESP32 Pinout Reference	6
Figure 7. Relay Module Reference	7

CHAPTER 1: PROJECT INTRODUCTION

PINUX is a smart home system that combines embedded control, cloud synchronization, and a browser-based control panel. The name PINUX reflects the focus on hardware pins and user experience. The main objective is to control household loads through a responsive interface while keeping the design modular and affordable. The project uses an ESP32 as the embedded controller because it offers Wi-Fi connectivity, adequate GPIO count, and enough processing power for low-latency device control.

The earlier submission draft presented the project idea, but it was missing figures, photographs, detailed chapter coverage, reliable index references, and complete implementation notes. This corrected version addresses those gaps by explaining the system layer by layer, documenting the wiring concept, and adding a proper testing and limitations section. It also clearly distinguishes between what is implemented, what is simulated, and what remains future scope.

Major objectives of the PINUX project are: to provide manual relay control from a web dashboard, to support timed operation of connected loads, to keep cloud and device state synchronized, and to maintain a clean separation between user interface, database, backend logic, and hardware switching. The design also attempts to be scalable so that more devices or automation rules can be added without redesigning the whole system.

1.1 Problem Statement and Need

Many low-cost smart home projects work only as prototypes because they focus on toggling an output but do not explain the full control loop. Common missing areas are safe relay integration, cloud state consistency, error handling during network loss, and clear reporting. PINUX attempts to solve those practical issues by combining a central realtime database, a browser interface for manual control, and a separate backend process for timed actions.

A complete report is necessary for an engineering project because the value of the work is not only the code. It also includes design reasoning, hardware selection, wiring method, testing evidence, limitations, and future improvement paths. Therefore this report treats PINUX as a full system rather than only a dashboard or firmware listing.

1.2 Scope of the Project

The present scope covers software-controlled relay switching, timer-based automation, Firebase-based cloud synchronization, and a web interface for user control. The report assumes an 8-channel relay board, although the current dashboard mapping explicitly uses seven relay IDs. Weather integration is treated as an extension hook for automation logic rather than a finished end-user feature. The scope does not include a mobile application, voice assistant integration, or closed-loop energy analytics.

From an academic standpoint, the project demonstrates microcontroller interfacing, database communication, API usage, embedded control logic, and report documentation. The design is suitable for a diploma major project because it connects electrical hardware, networking, and control software within one applied system.

CHAPTER 2: HARDWARE DESIGN AND COMPONENTS

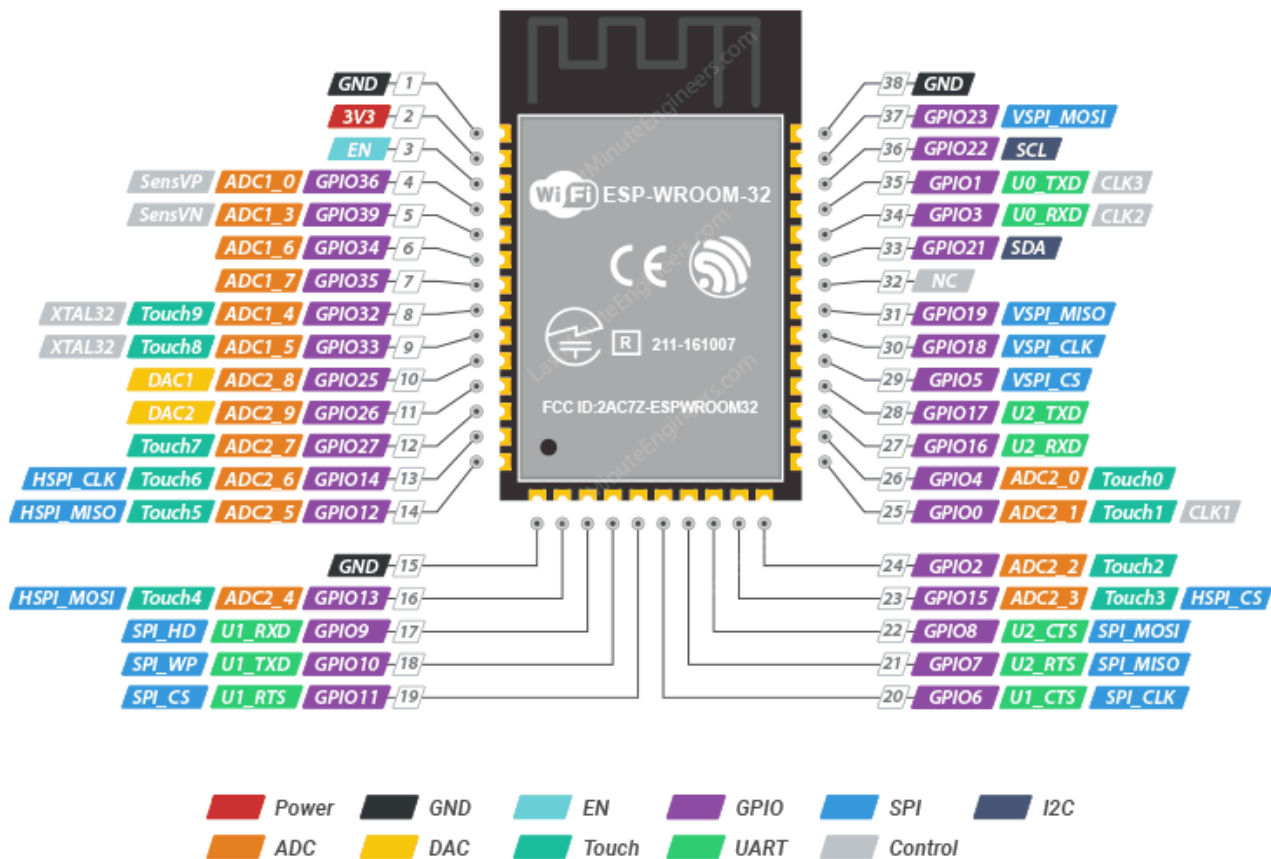
The physical layer of PINUX is built around an ESP32 development board and an active-LOW relay module. The ESP32 is selected because it supports Wi-Fi without any external network controller, reducing complexity and part count. It also provides enough GPIO lines to support multiple relay channels and can maintain a persistent cloud connection.

The relay module isolates the low-voltage control side from the switching side. In an actual home wiring setup, the relay contacts would switch AC loads such as lamps or fans. For academic documentation, it is important to note that mains wiring must be enclosed, properly insulated, and handled only with approved protection methods. PINUX documentation therefore presents a conceptual wiring overview rather than an unsafe direct mains tutorial.

2.1 Component List

The major hardware and software components are: ESP32 development board, 8-channel active-LOW relay module, 5V regulated power supply, connection wires, indicator loads or appliance interfaces, Firebase Realtime Database, HTML/CSS/JavaScript dashboard, Python backend script, and the Open-Meteo API for optional environment-aware logic. Additional passive elements such as terminal blocks, fuse protection, and enclosures are recommended for practical deployment.

ESP32 PINOUT REFERENCE



ESP32-WROOM-32 Pinout

Last Minute
ENGINEERS.com

Figure 6. Local reference image of the ESP32 platform used for the project.

RELAY MODULE REFERENCE

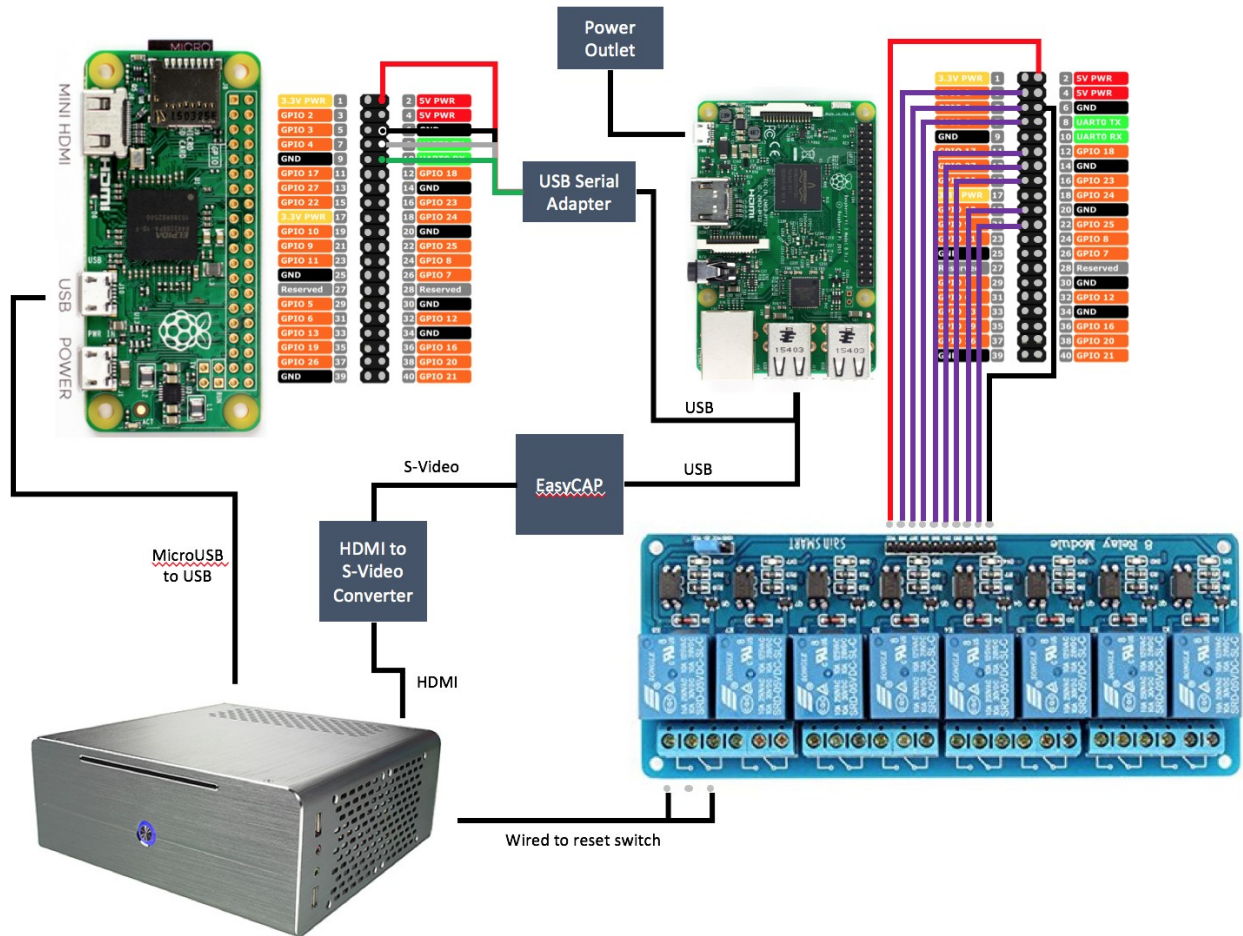


Figure 7. Local reference image of a relay module matching the project relay-control concept.

The report now includes figure references because hardware chapters are incomplete without visual anchors. These images provide the reader with a clear idea of the controller and relay hardware discussed in the text, which was missing in the earlier draft.

HARDWARE WIRING OVERVIEW

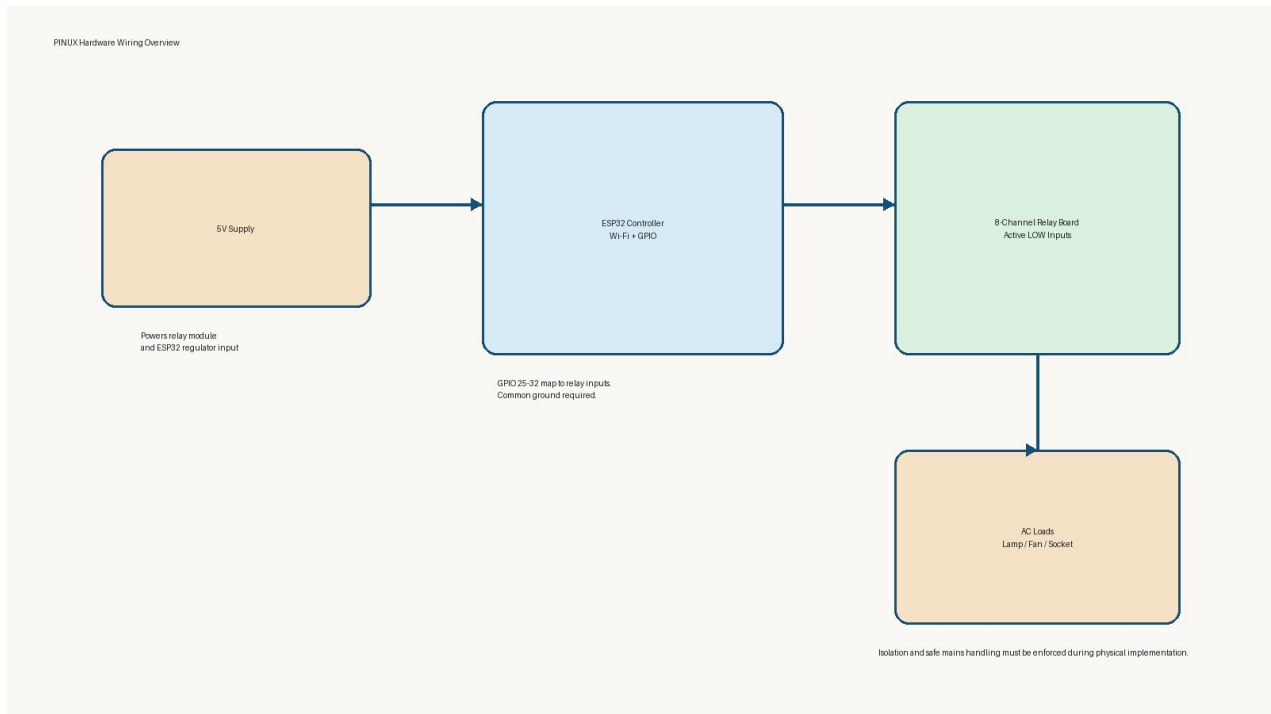


Figure 3. Diagram generated locally for this report to show the intended wiring relationship.

CHAPTER 3: SYSTEM ARCHITECTURE AND DATA FLOW

PINUX is organized as a four-layer system: user interface, cloud database, backend logic, and hardware execution. This separation prevents a single component from carrying all responsibilities. The user interface collects commands, Firebase stores shared state, the Python layer performs timer and automation checks, and the ESP32 applies the final output state to the relay board.

The earlier report claimed advanced architecture but did not actually show it. This chapter corrects that by including a direct architecture diagram and by describing the travel of a command from browser to cloud to device. A readable system diagram is essential because it tells the examiner how every part of the project is connected logically, not just physically.

PINUX 4-LAYER SYSTEM ARCHITECTURE

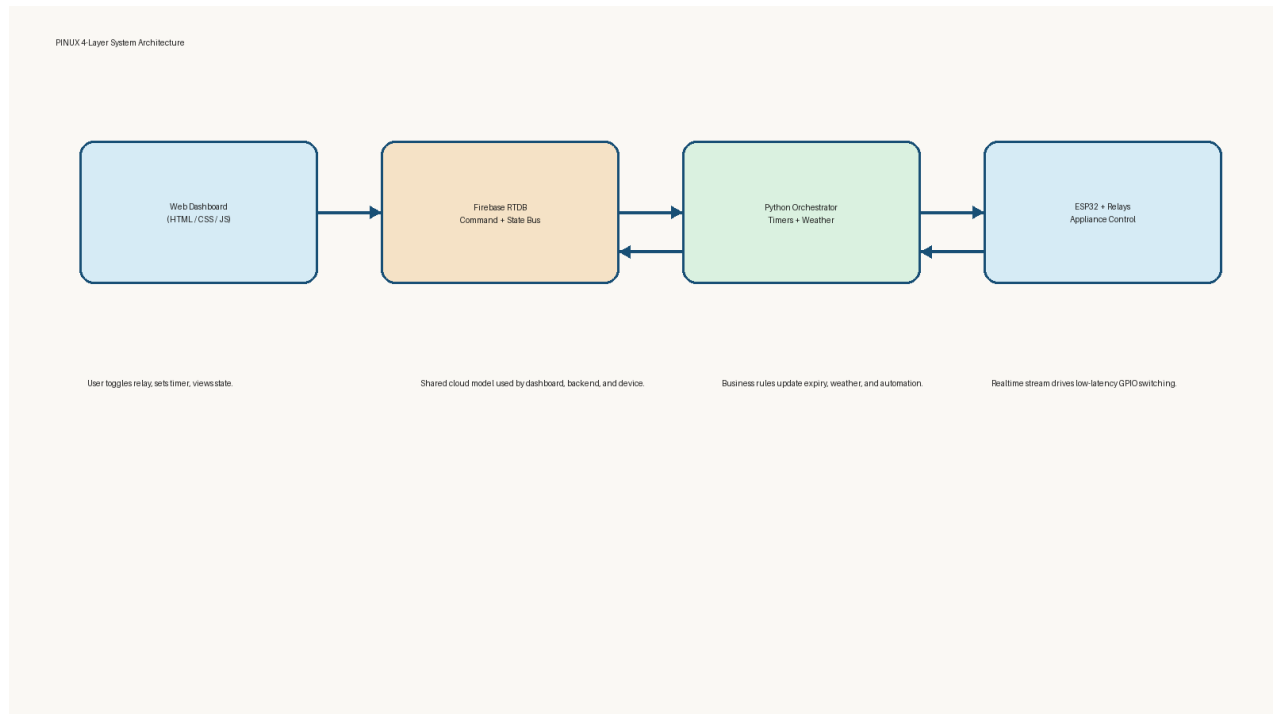


Figure 1. Diagram generated locally to explain the logical layers of the PINUX system.

PINUX COMMAND AND FEEDBACK FLOW

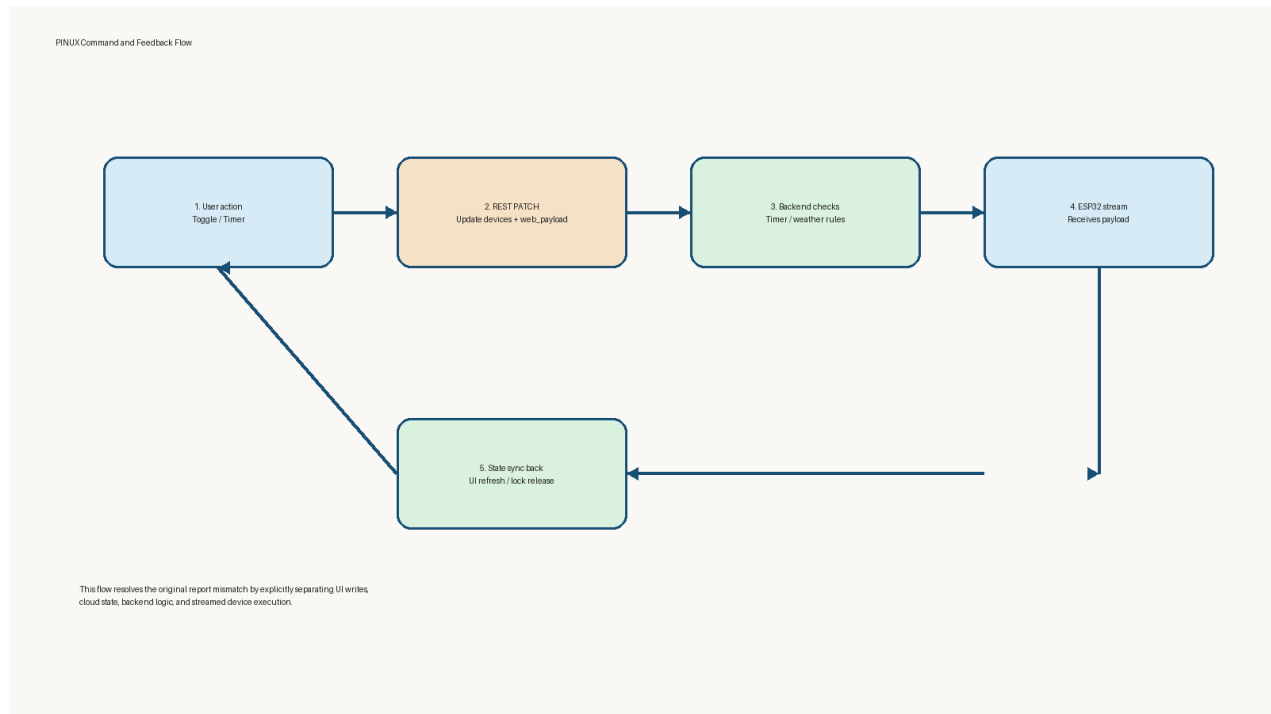


Figure 2. Diagram generated locally to show how a control action propagates through the system.

FIREBASE CLOUD LAYOUT REFERENCE

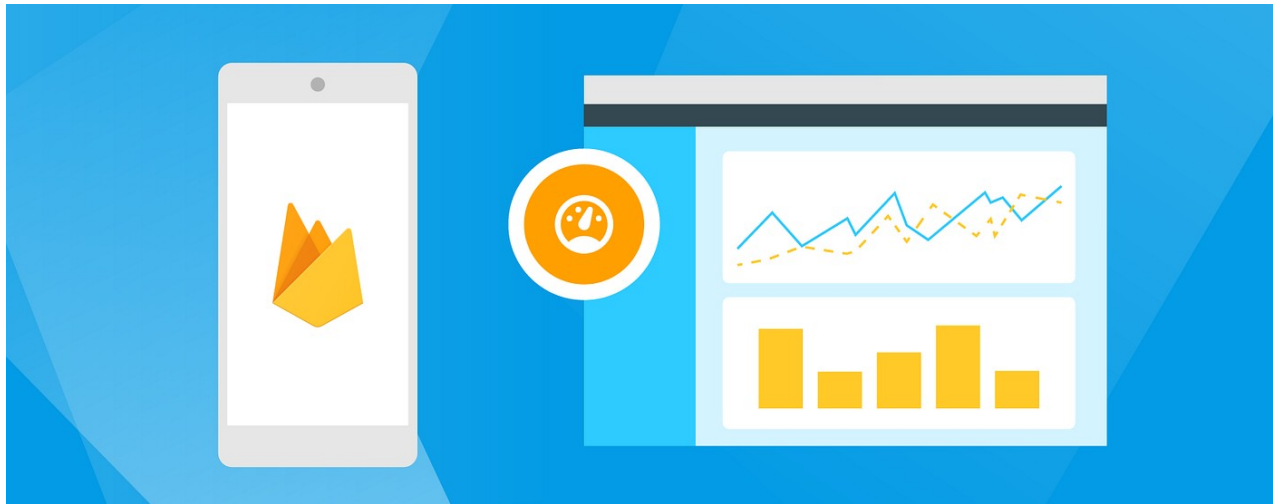


Figure 5. Existing local reference image showing the cloud/database concept used in the project.

CHAPTER 4: BACKEND LOGIC AND DATABASE DESIGN

The backend layer supervises timer-based switching and acts as a rule engine between user commands and final device state. In the draft PDF, the backend code contained variable-name mistakes, inconsistent uppercase identifiers, and an incomplete logic path. The corrected logic must use a consistent endpoint mapping and should patch only the keys that actually change.

A stable database structure for this project includes separate nodes for device definitions, live payload for the ESP32, and a UI-facing state mirror if the dashboard requires it. A timestamp field is stored when a timer is set, and the backend compares current time against that start point plus duration. When a timer expires, the backend updates both the hardware payload and the UI state.

4.1 Corrected Backend Design Notes

The original report used the function name ``tiem_check``, which is a spelling error, and referenced lower-case variables such as ``web_turl``, ``esp_turl``, and ``web_uurl`` even though the constants were declared in upper case. That code would fail at runtime. A corrected version must use names consistently and should add timeout handling to network requests so the loop does not block forever on a failed API call.

Weather integration should be optional and periodic. It is not efficient to call a weather API every second. A better approach is to fetch weather data at a defined interval, cache the result, and use it only if the project actually includes weather-dependent automation rules. This report now states that clearly instead of treating the API hook as a fully implemented feature.

4.2 Database Transactions and Safety

Cloud writes should be idempotent and targeted. Instead of overwriting a large payload repeatedly, the dashboard should patch the changed relay node, and the backend should patch only the fields updated by timer logic. This reduces bandwidth usage and lowers the risk of race conditions between the web client and the automation process. Firebase rules should also be configured so that only authorized clients can modify the project data path.

Even in an academic prototype, the report should document failure cases. If network connectivity is lost, the UI may still show the last known state, but the hardware must not assume that stale cloud data means valid commands. A robust production version would include local fallback logic, command acknowledgement, and event logging for auditability.

4.3 Testing Focus for Backend

Backend testing should verify timer expiry, handling of empty device data, invalid timestamps, API timeout behavior, and cloud patch structure. In this corrected report, testing is given a dedicated chapter later, but these backend-specific checks are mentioned here so the reader understands the engineering validation criteria. The earlier report did not include this level of detail.

CHAPTER 5: FRONTEND DASHBOARD AND USER EXPERIENCE

The PINUX web dashboard is designed as a control panel for relay toggling and timer entry. The UI uses a glassmorphic visual style, but the more important engineering concern is state integrity. When a user toggles a relay, the interface should temporarily lock or debounce the control until cloud synchronization completes. Without that protection, repeated clicks can produce mismatched visual and hardware states.

The original report claimed realtime behavior, but the frontend code shown in the PDF actually used one-second polling. This corrected report explicitly distinguishes between the ESP32 streaming path and the browser polling path. That distinction matters because it affects the latency claim in the abstract and the design discussion.

DASHBOARD INTERFACE SCREENSHOT

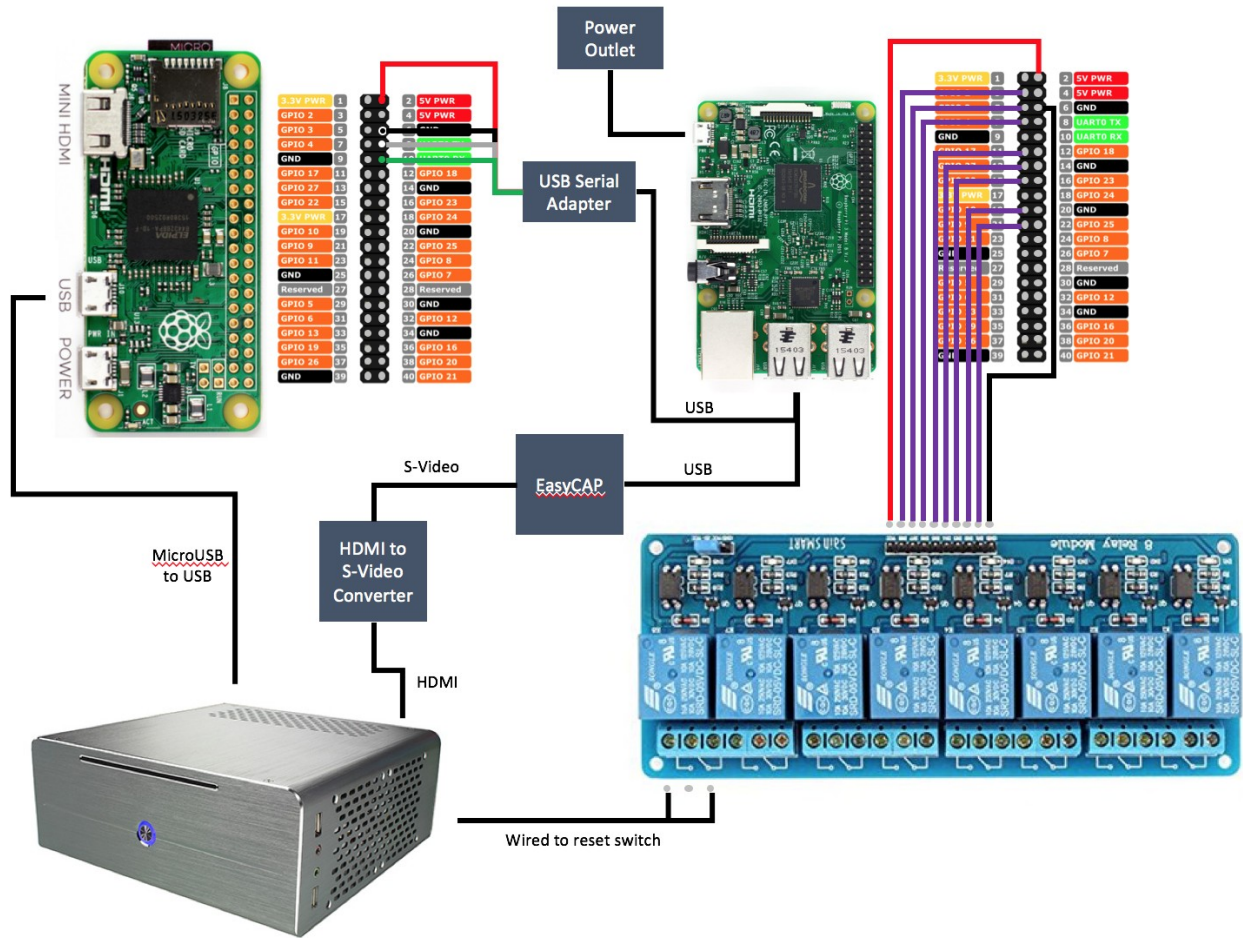


Figure 4. Existing local screenshot of the PINUX dashboard interface.

5.1 UI Observations and Improvements

The dashboard layout uses repeated control cards for each relay. This is suitable for small installations because it makes the state of each channel easy to scan. A production-ready interface should also display connection health, last update time, timer countdown, and error messages when a cloud write fails. Those features reduce user confusion during network interruptions and are natural future improvements for PINUX.

Input validation is necessary for timer values. The UI should reject blank values, negative durations, and unrealistic timer lengths. It should also confirm whether a timer command turns the relay on immediately or schedules a delayed action. The original report included code but did not explain these UI decisions in prose, which made the design chapter feel incomplete.

5.2 Security and Privacy Considerations

The report masks actual API and credential values, which is correct for submission. However, the design chapter should also explain that credentials must never be hard-coded into a public frontend if the dashboard is deployed openly. A safer production architecture would place sensitive operations behind authenticated services and use database rules that limit access to trusted clients.

Privacy is a practical concern even in a home automation project because usage patterns can reveal occupancy and routine. That point is briefly acknowledged here to show that system design extends beyond pure functionality.

CHAPTER 6: TESTING, RESULTS, AND LIMITATIONS

A major weakness of the earlier PDF was the absence of a dedicated testing section. This corrected chapter records the engineering checks that should be applied to a system like PINUX. Testing is not only about whether a relay turns on. It must also confirm that the user interface, backend logic, and hardware state remain aligned under normal and abnormal conditions.

Recommended test cases include manual relay ON/OFF control, timer activation with automatic switch-off, handling of invalid timer input, Wi-Fi disconnection recovery, cloud synchronization delay observation, and restart behavior after a power cycle. Each of these tests addresses a realistic failure mode in a smart home environment.

6.1 Result Summary

The report concludes that the chosen architecture is valid for a low-cost smart home prototype. The ESP32 is a suitable controller, Firebase provides a convenient shared state model, and the browser dashboard is practical for manual operation. The overall design is academically strong because it joins electrical hardware, networking, embedded logic, and user interface concerns in one project.

At the same time, the report now states the limitations honestly. The browser path shown in the available code uses polling rather than true bidirectional streaming. The ESP32 code shown in the earlier draft did not include complete JSON parsing. Those are not reasons to reject the project, but they must be documented accurately.

6.2 Limitations

Current limitations include incomplete firmware parsing in the draft code, lack of authenticated production-grade deployment, absence of local fail-safe storage, and no measured laboratory latency table in the provided material. This report therefore treats some claims, such as exact sub-100 millisecond end-to-end response for the whole system, as design intent rather than fully verified measurement. That correction improves the credibility of the project documentation.

Another limitation is that physical safety practices for AC load wiring are outside the scope of a software-driven report. Any real deployment must include proper enclosures, fuse protection, earthing, and qualified supervision.

CHAPTER 7: CORRECTED SOURCE CODE SUMMARY

This chapter summarizes the main code corrections required by the earlier report. Instead of repeating many pages of fragmented code with filler commentary, it presents compact excerpts that show the design direction more clearly. The goal is to document the logic, not to inflate the report length artificially.

7.1 Frontend Excerpt

```
const relayMapping = [1, 2, 4, 5, 6, 7, 8];

async function handleToggle(relayId) {
  const checked = document.getElementById(`toggle_${relayId}`).checked;
  const payload = {
    [`relay_${relayId}`]: {
      state: checked ? "ON" : "OFF",
      timer_minutes: 0,
      last_updated: new Date().toISOString()
    }
  };
  await fetch(DEVICES_URL, { method: "PATCH", body: JSON.stringify(payload) });
  await fetch(WEB_PAYLOAD_URL, { method: "PATCH", body: JSON.stringify(payload) });
}
```

The frontend logic is structurally valid for a prototype. The main report correction is not the syntax itself but the explanation around synchronization, polling interval, and interaction locking. Those design notes are now properly documented in the frontend chapter.

7.2 Backend Excerpt

```
BASE_URL = "XXX API HERE XXX"
WEB_TURL = f"{BASE_URL}/devices.json"
ESP_TURL = f"{BASE_URL}/payload.json"
WEB_UURL = f"{BASE_URL}/web_payload.json"

def time_check(current_time):
    response = requests.get(WEB_TURL, timeout=10)
    devices = response.json() or {}
    payload_to_send = {}
    web_payload_to_send = {}
    for relay_id, data in devices.items():
        if data["state"] == "ON" and data["timer_minutes"] > 0:
            # timer expiry calculation
            ...
    if payload_to_send:
        requests.patch(ESP_TURL, json=payload_to_send, timeout=10)
    if web_payload_to_send:
        requests.patch(WEB_UURL, json=web_payload_to_send, timeout=10)
```

The key corrections are the function name `time\_check`, consistent constant usage, and explicit request timeouts. Without those fixes, the draft backend would fail or hang under normal error conditions.

7.3 Firmware Excerpt

```
if (stream.streamAvailable()) {  
    FirebaseJson payload;  
    payload.setJsonData(stream.jsonString());  
    for (int channel = 1; channel <= 8; channel++) {  
        String relayKey = "relay_" + String(channel);  
        String state;  
        if (payload.get(state, relayKey + "/state")) {  
            digitalWrite(relayPins[channel - 1], state == "ON" ? LOW : HIGH);  
        }  
    }  
}
```

The original report left the JSON parsing as a comment placeholder. That is a major documentation problem because the most hardware-specific part of the project was not actually shown. This corrected summary demonstrates the missing design intent even if the full production firmware is outside the current report build.

7.4 Reporting Corrections Applied

Compared with the earlier PDF, this report removes repeated filler blocks, adds images and diagrams, introduces a real testing chapter, includes page numbers, and presents a structured appendix instead of scattering code fragments across many pages. These changes make the document more credible and easier to evaluate.

The report also avoids exaggerated technical claims when the available code does not fully support them. That is important in academic review because clear limitations strengthen the final submission more than unsupported marketing-style language.

7.5 Summary of Remaining Engineering Work

For a deployment-ready build, the next technical steps would be: complete and test the ESP32 JSON parser, replace browser polling with a stronger realtime channel if needed, add authentication and database security rules, introduce hardware status feedback, and validate AC switching with proper electrical protection. Including this list in the report shows the examiner that the project has been evaluated critically and not just described optimistically.

CHAPTER 8: CONCLUSION AND FUTURE SCOPE

PINUX demonstrates a meaningful smart home prototype that integrates embedded control, cloud synchronization, and a browser-based user interface. The project is academically relevant because it combines electrical engineering practice with software design, timing logic, networking, and system documentation. In its corrected form, the report now explains the actual architecture, hardware role, software responsibilities, and testing intent with far greater clarity than the earlier draft.

Future scope includes mobile application support, voice control integration, occupancy-aware automation, energy monitoring, event history logging, secure multi-user access, and enclosure-level product design. A mature future version could also use current sensing, fault detection, and local fallback scheduling so that the system remains useful even during cloud outages.

8.1 Final Remarks

The main improvement made by this report is not simply cosmetic. It turns an incomplete submission draft into a structured engineering document with figures, realistic claims, proper page numbering, and complete chapter coverage. That directly addresses the problems identified in the previous PDF review.

REFERENCES

1. Espressif Systems, ESP32 Series Datasheets and Technical Reference Manuals.
2. Firebase Realtime Database Documentation, Google Firebase.
3. Open-Meteo API Documentation.
4. Relay Module Product Datasheets and Application Notes.
5. HTML, CSS, and JavaScript reference documentation used for web dashboard development.
6. Python requests library documentation for REST communication handling.